FPTAPTECH EDUCATION SERIES

API-First Development

Xây dựng REST API chuẩn doanh nghiệp với Spring Boot và OpenAPI 3.0 Specification

Developed by FPTAPTECH Education

CORE	LANG	SPEC
Spring Boot	**Java LTS**	**OpenAPI**
v3.4.x	Version 21	v3.0.3



FPTAPTECH

Giới thiệu API-First Development

CONCEPTS & WORKFLOW

Khái niệm

API-First là phương pháp tiếp cận coi **"Hợp đồng API" (API Contract)** là công dân hạng nhất. Tài liệu đặc tả API (như OpenAPI/Swagger) được định nghĩa và thống nhất **trước khi** viết bất kỳ dòng code nào.

Tại sao chọn API-First?

Fail-Fast & Giảm rủi ro
Phát hiện lỗi thiết kế ngay từ giai đoạn bàn giấy, tránh việc sửa code tốn kém sau này.

Giao tiếp rõ ràng
OAS đóng vai trò là "ngôn ngữ chung" giữa Developers, Testers, FE/Mobile và Stakeholders.

Phát triển song song
Frontend có thể dùng Mock server để code ngay lập tức, không cần đợi Backend hoàn thành.



Quy trình làm việc

1. **Design OAS**
   Viết file YAML/JSON mô tả API
2. **Review**
   Team thống nhất & chỉnh sửa
3. **Generate Code**
   Sinh code Server & Client SDK
4. **Implement**
   Viết Business Logic vào Controller

SPECIFICATION

OpenAPI 3.0

Chuẩn mô tả API RESTful độc lập với ngôn ngữ lập trình. OAS định nghĩa tài nguyên, phương thức, tham số và mô hình dữ liệu trong một file duy nhất.

HỆ SINH THÁI CÔNG CỤ

Swagger Editor
Thiết kế & Preview tài liệu API (YAML/JSON)

OpenAPI Generator
Tự động sinh code Server Stubs & Client SDKs

Swagger UI
Giao diện tương tác trực quan cho người dùng

Official Spec: github.com/OAI/OpenAPI-Specification

shop-api.yaml

```yaml
openapi: 3.0.3
info:
  title: Shop API
  description: API quản lý đơn hàng demo
  version: 1.0.0

servers:
  - url: http://localhost:8080/api/v1
    description: Local Development Server

paths:
  /health:
    get:
      summary: System health check
      tags:
        - system
      responses:
        '200':
          description: OK - Service is running
          content:
            application/json:
              schema:
                type: object
                properties:
                  status:
                    type: string
```

FPTAPTECH

main 0 errors YAML UTF-8

COMPARISON

Phương pháp Phát triển

Tại sao nên chuyển đổi từ Code-First sang API-First?

Truyền thống

CODE-FIRST APPROACH

Code → API → Doc

Phản hồi chậm trễ

Stakeholders chỉ thấy API khi code đã xong. Việc sửa đổi thiết kế lúc này rất tốn kém thời gian (Rework).

Tài liệu lệch pha

Documentation thường được viết sau cùng, dễ bị outdated và không khớp với implementation thực tế.

Quy trình tuần tự (Serial)

Frontend/Mobile bị chặn (blocked) cho đến khi Backend hoàn thành API.

VS

Hiện đại RECOMMENDED

API-FIRST APPROACH

Contract → Code + Tests

Review thiết kế sớm

Thống nhất hợp đồng (Contract) trước khi code. Phát hiện lỗi logic ngay trên giấy ("Fail-Fast").

Làm việc song song

Sau khi có file OAS, Frontend dùng Mock Server để code ngay lập tức, không cần chờ Backend.

Single Source of Truth

Code, Document và Test cases được sinh tự động từ một nguồn duy nhất (File YAML).

EDUCATION SERIES
FPTAPTECH

ARCHITECTURE OVERVIEW

Kiến trúc Hệ thống Demo

Java 21 | Spring Boot 3.4

Clients

Swagger UI / curl / Postman

Spring Boot Application

API

Customer Module

- CustomersApi
- CustomerController

API

Product Module

- ProductsApi
- ProductController

API

Order Module

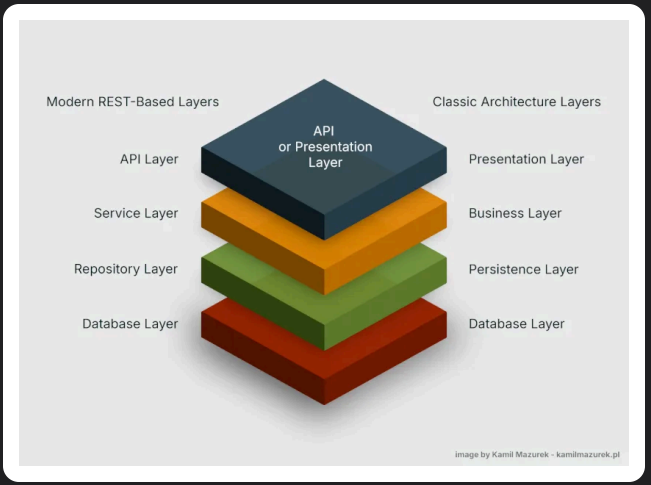
- OrdersApi
- OrderController

Service Layer (Business Logic)

Repository Layer

In-Memory ConcurrentHashMap

REFERENCE ARCHITECTURE



Standard Layered Architecture Pattern

Phạm vi Demo

- **CRUD Khách hàng**
  Tạo mới, xem danh sách, chi tiết
- **Quản lý Sản phẩm**
  Tạo, cập nhật kho, giá bán
- **Xử lý Đơn hàng**
  Tạo đơn, tính tiền, lịch sử

Module Khách hàng

Customer Management API

Path Prefix: **/api/v1**

API ENDPOINTS

GET `/customers`

Liệt kê khách hàng trong hệ thống.

POST `/customers`

Tạo mới. Body JSON required.

GET `/customers/{id}`

Chi tiết theo ID. 404 nếu null.

Validation Rules

- `name`: Required, 2-100 chars
- `email`: Valid format & Unique

DATA MODELS (JAVA 21 RECORDS)

Customer.java

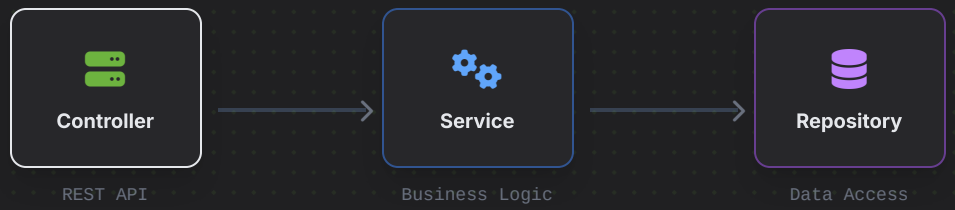
```java
package com.fptaptech.demo.model;
import jakarta.validation.constraints.*;

public record NewCustomer(
    @NotNull @Size(min=2, max=100) String name,
    @NotNull @Email String email
) {}

public record Customer(
    @NotNull Long id,
    @NotNull String name,
    @NotNull String email
```

DTOs

LAYERED ARCHITECTURE FLOW



FPTAPTECH

Module Sản phẩm

Product Management API

Path Prefix: **/api/v1**

API ENDPOINTS

GET `/products`

Truy xuất danh mục tất cả sản phẩm.

POST `/products`

Thêm sản phẩm mới vào kho hàng.

GET `/products/{id}`

Xem chi tiết giá và tồn kho của một sản phẩm.

DATA MODELS (JAVA 21 RECORDS)

Product.java

```java
package com.example.demo.model;

import java.math.BigDecimal;
import jakarta.validation.constraints.*;

/**
 * DTO để thêm sản phẩm mới
 */
public record NewProduct(
    @NotNull
    String name,

    @NotNull
    @PositiveOrZero
    BigDecimal price,

    @NotNull
    @Min(0)
    Integer stock
) {}
```

FPTAPTECH

Module Đơn hàng

Order Processing API

Path Prefix: **/api/v1**

API ENDPOINTS

POST `/orders`

Tạo đơn hàng mới. Tính toán tổng tiền tự động từ items.

GET `/orders/{id}`

Lấy thông tin chi tiết đơn hàng và trạng thái hiện tại.

GET `/orders?customerId=...`

Liệt kê đơn hàng theo khách hàng. Query param bắt buộc.

Business Rules

- `items`: Danh sách không được rỗng
- `quantity`: Phải lớn hơn 0
- `status`: Mặc định là NEW khi tạo

DATA MODELS & ENUMS

Order.java / OrderStatus.java

```java
package com.example.demo.model;

import java.util.List;
import java.math.BigDecimal;

/** Trạng thái đơn hàng */
public enum OrderStatus {
    NEW, PAID, CANCELLED
}

/** Chi tiết từng item trong đơn hàng */
public record OrderItem(
    Long productId,
    Integer quantity,
    BigDecimal unitPrice
) {}

/** Request tạo đơn hàng mới */
public record NewOrder(
```

COMPLEX TYPES

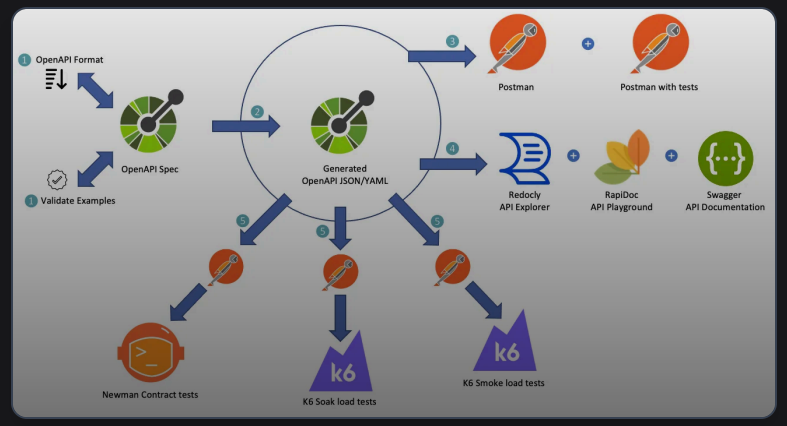
FPTAPTECH

API CONTRACT

Thiết kế Contract

Bản hợp đồng duy nhất giữa các bên. Quy trình Spec-Driven Development đảm bảo tính nhất quán trước khi viết code.

WORKFLOW DIAGRAM Spec-First Approach



Mô hình phát triển dựa trên đặc tả (Spec-Driven Development)

Format: YAML (Standard for Spring Boot)

src/main/resources/shop-api.yaml

RO Protected

```yaml
openapi: 3.0.3
info:
  title: Shop API (Customers, Products, Orders)
  version: 1.0.0
servers:
  - url: http://localhost:8080/api/v1

tags:
  - name: customers
  - name: products
  - name: orders

paths:
  /customers:
    get:
      tags: [customers]
      summary: List all customers
      responses:
        '200':
          description: OK
          content:
            application/json:
              schema:
                type: array
                items: { $ref: '#/components/schemas/Customer' }
    post:
      tags: [customers]
```

FPTAPTECH

api-design Valid OAS 3.0 YAML Spaces: 2 UTF-8

CONFIGURATION

Cấu hình Maven

Thiết lập `pom.xml` để tích hợp Spring Boot 3.4.x, các dependencies cần thiết và cấu hình plugin sinh code tự động.

THÀNH PHẦN CHÍNH

Environment

Java 21 LTS & Spring Boot 3.4.13 làm nền tảng phát triển.

Dependencies

Web, Validation và `springdoc-openapi` để hiển thị Swagger UI.

Generator Plugin

Cấu hình quan trọng:

```
useSpringBoot3: true
interfaceOnly: true
useJakartaEe: true
```

Plugin sẽ chạy trong phase `generate-sources`

pom.xml

```xml
<properties>
    <java.version>21</java.version>
    <spring-boot.version>3.4.13</spring-boot.version>
    <openapi.gen.version>7.6.0</openapi.gen.version>
</properties>

<dependencies>
    <!-- Web & Validation -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <!-- Swagger UI Runtime -->
    <dependency>
        <groupId>org.springdoc</groupId>
        <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
        <version>2.6.0</version>
    </dependency>
</dependencies>

<build>
  <plugins>
```

FPTAPTECH

main Ready XML UTF-8

AUTOMATION / Workflow

Quy trình Sinh Code Tự động

Từ bản thiết kế OpenAPI đến Server Skeleton hoàn chỉnh

codegen_status: active

1 Lưu đặc tả API Contract

Lưu file thiết kế từ Swagger Editor vào thư mục resources.

`src/main/resources/shop-api.yaml`

2 Chạy Maven Goal

Kích hoạt plugin `openapi-generator` để sinh code.

```
$ mvn clean generate-sources
```

3 Kiểm tra Artifacts

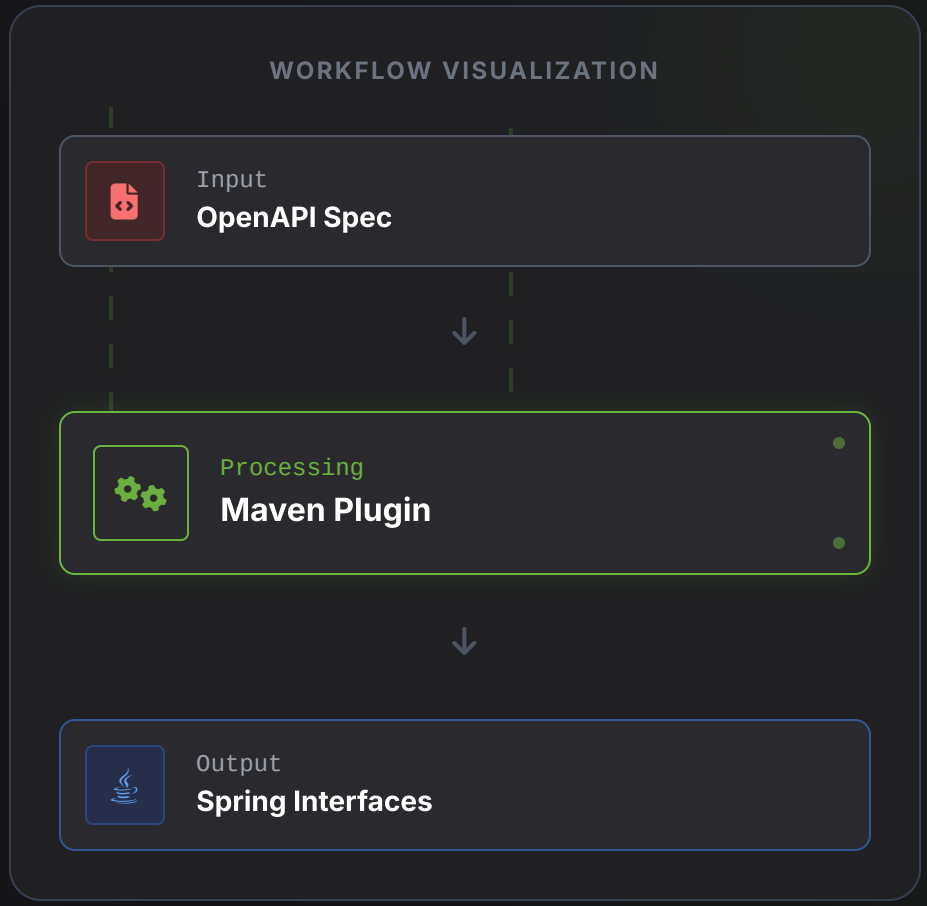
Kiểm tra thư mục `target/generated-sources`.

`CustomersApi` `ProductsApi` `Models`

✓ Implement Controller

Viết business logic vào Controller.

```
implements CustomersApi
```



Automated mapping from Schema to Java Po[illegible]

AUTOMATED WORKFLOW EFFICIENCY

FPTAPTECH
EDUCATION SERIES

SERVER SIDE

Implementation Logic

Hiện thực hóa các Interface đã được sinh tự động. Tập trung hoàn toàn vào business logic thay vì cấu hình boilerplate.

THÀNH PHẦN CHÍNH

Interface Generated

Class `CustomersController` implement `CustomersApi` để đảm bảo tuân thủ 100% đặc tả OpenAPI.

In-Memory Store

Sử dụng `ConcurrentHashMap` để mô phỏng Database, đảm bảo thread-safe cho demo đơn giản.

ResponseEntity

Spring Wrapper trả về HTTP Status Code chuẩn (200 OK, 201 Created, 404 Not Found).

Java 21 + Spring Boot 3.4.x

CustomersController.java

```java
@RestController
@RequiredArgsConstructor
public class CustomersController implements CustomersApi {

    // In-memory storage simulation
    private final Map<Long, Customer> store = new ConcurrentHashMap<>();
    private final AtomicLong seq = new AtomicLong(1);

    @Override
    public ResponseEntity<List<Customer>> listCustomers() {
        return ResponseEntity.ok(new ArrayList<>(store.values()));
    }

    @Override
    public ResponseEntity<Void> createCustomer(NewCustomer body) {
        long id = seq.getAndIncrement();
        // Create Customer from generated record/POJO
        Customer newCustomer = new Customer();
        newCustomer.setId(id);
        newCustomer.setName(body.getName());
        newCustomer.setEmail(body.getEmail());

        store.put(id, newCustomer);
        return ResponseEntity.status(201).build();
    }

    @Override
    public ResponseEntity<Customer> getCustomer(Long id) {
```

FPTAPTECH

main 0 errors Java UTF-8

LIVE DEMO / E2E Testing

Demo Hoàn Chỉnh & Test API

Kiểm thử luồng nghiệp vụ từ Khởi chạy đến Tạo đơn hàng

SERVER READY | Port: 8080

1 Khởi chạy Server

```
$ mvn spring-boot:run
```

2 Tạo Customer

```
curl -X POST /api/v1/customers \
-d '{"name":"An Dev", "email":"an@test.com"}'
```

3 Tạo Product

```
curl -X POST /api/v1/products \
-d '{"name":"Laptop Pro", "price":2000, "stock":10}'
```

4 Tạo Order

```
curl -X POST /api/v1/orders \
-d '{"customerId":1, "items":[{"productId":1,"qty":1}]}'
```

5 Kiểm tra Order

```
curl -X GET /api/v1/orders/1
```

200 OK

6 Swagger UI

http://localhost:8080/swagger-ui.html

✓ Content-Type: application/json ✓ Accept: application/json

FPTAPTECH
EDUCATION SERIES

Kết luận & Next Steps

SUMMARY & FUTURE ROADMAP

Tổng kết

API-First Development giúp tăng tốc độ phát triển, giảm thiểu rủi ro sai lệch yêu cầu và chuẩn hoá giao tiếp giữa các team thông qua **OpenAPI Contract**.

Kết quả đạt được

YAML Specification

Thiết kế hoàn chỉnh API Contract cho 3 modules: Customers, Products, Orders.

Code Generation

Tự động sinh Server Stub & DTOs chuẩn Spring Boot 3.4.x / Java 21.

Implementation & Demo

Xây dựng Controller logic và chạy thử nghiệm thành công với Swagger UI/Curl.

Các bước tiếp theo

1. **Robustness**
   Thêm Validation & Global Error Handling
2. **Persistence & Test**
   Kết nối JPA/DB & Viết Unit/Integration Tests
3. **Security**
   Tích hợp Spring Security (JWT/OAuth2)
4. **Ops & Automation**
   CI/CD Client SDK, Versioning & Observability